

Laravel Interview Preparation Guide

(हिंदी में)

Routing से लेकर Real-World Scenario Questions तक — 5 वर्ष अनुभव के लिए
एक संपूर्ण गाइड

24 Topics • Question & Answer Format • Real-World Scenarios शामिल

विषय सूची (Table of Contents)

1. Routing & Middleware	5 Q&A
2. MVC & Request Lifecycle	4 Q&A
3. Eloquent ORM & Relationships	6 Q&A
4. MySQL & Database Optimization	5 Q&A
5. Authentication & Authorization	5 Q&A
6. Spatie Permission / RBAC	4 Q&A
7. REST API & Sanctum	5 Q&A
8. Validation & Form Requests	4 Q&A
9. Events & Listeners	4 Q&A
10. Jobs & Queues	5 Q&A
11. Redis & Cache	4 Q&A
12. Laravel Scheduler & Cron	3 Q&A
13. Service Container & Dependency Injection	4 Q&A
14. Service Provider & Facades	4 Q&A
15. Transactions & Exception Handling	4 Q&A
16. N+ 1 Query & Eager Loading	4 Q&A
17. Payment Gateway / Webhooks	4 Q&A
18. File Upload & Storage	4 Q&A
19. Security	5 Q&A
20. Git	4 Q&A
21. Laravel Performance Optimization	4 Q&A
22. Real-world Scenario Questions	4 Q&A
23. 5-Year Experience Tricky Questions	5 Q&A
24. Project-Based Questions	4 Q&A

1. Routing & Middleware

प्रश्न 1: Route और Middleware में क्या फर्क है?

उत्तर: Route यह तय करता है कि कौन सा URL किस Controller/Closure को call करेगा। Middleware, request और response के बीच एक filter/layer होता है जो request को Controller तक पहुँचने से पहले (या response को user तक जाने से पहले) intercept करता है — जैसे auth check, CORS, logging आदि।

प्रश्न 2: Route model binding क्या है?

उत्तर: Route model binding से आप Route parameter (जैसे {user}) को सीधे Eloquent model instance में automatically resolve करवा सकते हैं, बिना manually find() call किए। Implicit binding {user} से User::find(\$id) खुद हो जाता है, Explicit binding Route::bind() से customize होता है।

प्रश्न 3: Custom Middleware कैसे बनाते और register करते हैं?

उत्तर: php artisan make:middleware CheckRole से middleware बनता है, handle() method में logic लिखते हैं। फिर app/Http/Kernel.php में \$routeMiddleware (Laravel 10 से पहले) या bootstrap/app.php (Laravel 11+) में alias register करके route पर ->middleware('checkrole') लगाते हैं।

प्रश्न 4: Middleware groups और priority कैसे काम करती है?

उत्तर: web और api groups predefined middleware का set होते हैं। जब कई middleware एक साथ लगे हों तो उनका execution order Kernel में \$middlewarePriority array से decide होता है ताकि जैसे auth हमेशा throttle से पहले चले।

प्रश्न 5: Route caching production में क्यों जरूरी है?

उत्तर: php artisan route:cache सभी routes को एक single file में compile कर देता है जिससे route registration की speed बहुत बढ़ जाती है। ध्यान रहे Closure based routes cache नहीं होते, इसलिए production routes हमेशा Controller based होने चाहिए।

2. MVC & Request Lifecycle

प्रश्न 1: Laravel में एक request की पूरी lifecycle समझाइए।

उत्तर: Request सबसे पहले `public/index.php` में आती है, वहाँ से `bootstrap/app.php` Application बनाता है। फिर HTTP Kernel request को handle करता है — bootstrappers (config load, error handling आदि) चलते हैं, फिर global middleware, route matching, route middleware, Controller method call होता है और Response वापस भेजा जाता है, जो `terminate()` middleware से गुजरता है।

प्रश्न 2: Model, View, Controller की जिम्मेदारी क्या-क्या है?

उत्तर: Model database और business data को represent करता है (Eloquent)। View presentation layer है (Blade templates)। Controller बीच का layer है जो request लेकर Model से data लाता है और सही View लौटाता है — यानी logic को data और presentation से अलग रखना।

प्रश्न 3: Service Container request lifecycle में कहाँ fit होता है?

उत्तर: Application boot होते समय Service Providers register होते हैं जो bindings डालते हैं। जब भी कोई class (Controller, Job आदि) की जरूरत होती है, container automatically dependencies resolve करके inject करता है — यही IoC (Inversion of Control) है।

प्रश्न 4: Kernel.php में bootstrappers क्या काम करते हैं?

उत्तर: ये classes environment load करना, configuration cache करना, exception handler set करना, facades register करना और service providers boot करना जैसे शुरुआती काम करते हैं, request आगे बढ़ने से पहले।

3. Eloquent ORM & Relationships

प्रश्न 1: Eloquent Relationships के types बताइए।

उत्तर: hasOne, hasMany, belongsTo, belongsToMany (many-to-many, pivot table के साथ), hasManyThrough, hasOneThrough, morphOne/morphMany/morphToMany (polymorphic relationships) — ये मुख्य types हैं। हर relationship एक method के रूप में define होती है जो query builder return करती है।

प्रश्न 2: hasMany और belongsToMany में क्या फर्क है?

उत्तर: hasMany में foreign key दूसरी table में होती है (one-to-many, जैसे एक User के कई Posts)। belongsToMany many-to-many के लिए है और बीच में एक pivot table (जैसे role_user) रखती है जो दोनों तरफ की IDs स्टोर करती है।

प्रश्न 3: Polymorphic relationship कब इस्तेमाल करते हैं, उदाहरण दीजिए।

उत्तर: जब एक ही relationship multiple models से जुड़नी हो — जैसे Comment, Post और Video दोनों पर हो सकता है। Comment table में commentable_id और commentable_type columns रखकर morphTo/morphMany से यह achieve होता है, बिना अलग-अलग tables बनाए।

प्रश्न 4: Accessor और Mutator क्या हैं (Laravel 9+ Attribute syntax)?

उत्तर: Accessor attribute पढ़ते समय value को transform करता है, Mutator save करते समय। नया तरीका है protected function name(): Attribute { return Attribute::make(get: fn(\$v)=>..., set: fn(\$v)=>...); } — पुराना तरीका getNameAttribute()/setNameAttribute() था।

प्रश्न 5: Model Events (creating, saving, deleted आदि) कैसे use करते हैं?

उत्तर: Model में static::creating(function(\$model){...}) boot() method में लिख सकते हैं, या Observer class बनाकर php artisan make:observer से register कर सकते हैं। ये events किसी भी lifecycle action पर automatically trigger होते हैं जैसे hooks की तरह।

प्रश्न 6: Soft Deletes कैसे काम करता है?

उत्तर: Model में SoftDeletes trait और table में deleted_at column होने पर delete() record को actually delete नहीं करता, बस timestamp set करता है। Query default इन्हें exclude करती है, withTrashed() या onlyTrashed() से include/filter कर सकते हैं।

4. MySQL & Database Optimization

प्रश्न 1: Index कब और क्यों लगाना चाहिए?

उत्तर: जिन columns पर बार-बार WHERE, JOIN, ORDER BY होता है उन पर index लगाना चाहिए (जैसे foreign keys, email)। Index read speed बढ़ाता है पर insert/update थोड़ा धीमा करता है और storage भी लेता है, इसलिए हर column पर index लगाना सही नहीं है।

प्रश्न 2: Composite index किस situation में मदद करता है?

उत्तर: जब query में एक साथ कई columns पर filter/sort होता है (जैसे WHERE user_id = ? AND status = ?), तब composite index (user_id, status) single-column indexes से ज्यादा effective होता है। Column order query pattern (leftmost prefix rule) के हिसाब से decide होता है।

प्रश्न 3: Slow query को कैसे debug/optimize करेंगे?

उत्तर: EXPLAIN query लगाकर देखें कि index use हो रहा है या full table scan। MySQL slow query log enable कर सकते हैं। बड़ी tables पर SELECT * से बचें, जरूरी columns ही चुनें, pagination use करें, और N+ 1 problems check करें।

प्रश्न 4: Database Normalization vs Denormalization कब चुनेंगे?

उत्तर: Normalization (3NF तक) data redundancy कम करता है, writes consistent रहते हैं — transactional systems के लिए अच्छा। Denormalization (कुछ duplicate data रखना) reads तेज़ करता है — reporting/dashboard जैसे read-heavy systems में इस्तेमाल होता है।

प्रश्न 5: Migration में foreign key constraints कैसे handle करते हैं?

उत्तर: \$table->foreignId('user_id')->constrained()->onDelete('cascade') जैसे syntax से relationship और delete behavior define होता है। Production में large tables पर migration चलाते समय downtime का ध्यान रखना पड़ता है।

5. Authentication & Authorization

प्रश्न 1: Authentication और Authorization में फर्क क्या है?

उत्तर: Authentication यह verify करता है कि user कौन है (login/password check)। Authorization यह तय करता है कि authenticated user को क्या करने की permission है (Gates, Policies)।

प्रश्न 2: Laravel में Guards और Providers क्या करते हैं?

उत्तर: Guard यह define करता है कि user कैसे authenticate होगा request-by-request (जैसे session guard, sanctum guard)। Provider यह बताता है कि user data कहाँ से आएगा (Eloquent model या custom database table) — config/auth.php में दोनों configure होते हैं।

प्रश्न 3: Gates और Policies में क्या फर्क है?

उत्तर: Gate एक simple Closure-based authorization rule होता है, AuthServiceProvider में Gate::define() से बनता है — छोटे checks के लिए। Policy एक पूरी class होती है जो एक specific Model से जुड़ी होती है (जैसे PostPolicy) और उस model पर CRUD-level authorization रखती है — बड़े/complex logic के लिए बेहतर।

प्रश्न 4: \$this->authorize() और @can Blade directive कैसे use करते हैं?

उत्तर: Controller में \$this->authorize('update', \$post) policy check करता है और fail होने पर 403 देता है। Blade में @can('update', \$post) ... @endcan से conditionally UI elements दिखाते/छुपाते हैं।

प्रश्न 5: Multi-guard authentication (जैसे Admin और User अलग-अलग) कैसे implement करेंगे?

उत्तर: config/auth.php में अलग guard (admin) और provider define करते हैं, अलग table/model (Admin) use करते हैं, और routes पर middleware('auth:admin') लगाते हैं। Sessions भी अलग रहती हैं ताकि दोनों एक साथ independently login रह सकें।

6. Spatie Permission / RBAC

प्रश्न 1: Spatie laravel-permission package क्या solve करता है?

उत्तर: यह Role-Based Access Control (RBAC) आसान बनाता है — roles और permissions को database driven तरीके से manage करने देता है, बिना हर बार code/migration बदले नए roles/permissions add किए जा सकते हैं।

प्रश्न 2: Role और Permission में क्या फर्क है इस package में?

उत्तर: Permission एक specific action होता है (जैसे 'edit posts')। Role permissions का एक group/collection होता है (जैसे 'Editor' role में 'edit posts', 'delete posts' permissions हों)। User को directly permission भी assign हो सकती है या role के through।

प्रश्न 3: \$user->hasRole() और \$user->can() कैसे काम करते हैं?

उत्तर: hasRole('admin') सिर्फ role check करता है। can('edit posts') Laravel के authorization system से integrate होकर देखता है कि user के पास direct permission है या उसके role में वो permission है — दोनों cases handle करता है।

प्रश्न 4: बड़े applications में roles/permissions को कैसे structure करेंगे (real scenario)?

उत्तर: आमतौर पर seeder से base roles (Admin, Manager, Editor, Viewer) और permissions बनाते हैं, middleware ('role:admin' या 'permission:edit-posts') routes पर लगाते हैं, और Blade/@can से UI control करते हैं। Cache भी important है क्योंकि permission checks बार-बार होते हैं — package internally cache करता है, role/permission बदलने पर cache clear करना पड़ता है (permission:cache-reset)।

7. REST API & Sanctum

प्रश्न 1: REST API design के best practices क्या हैं?

उत्तर: Resource-based URLs (`/posts`, `/posts/{id}`), सही HTTP methods (GET, POST, PUT/PATCH, DELETE), सही status codes (200, 201, 404, 422, 500), versioning (`/api/v1/...`), और consistent response structure (Resource/Collection classes) — ये सब important हैं।

प्रश्न 2: Sanctum और Passport में क्या फर्क है, कब क्या use करें?

उत्तर: Sanctum lightweight है — SPA authentication (cookie based) और simple API token authentication दोनों के लिए, छोटे/medium projects के लिए काफी है। Passport पूरा OAuth2 server implement करता है — जब third-party apps को token-based access देना हो (जैसे 'Login with YourApp') तब Passport चाहिए।

प्रश्न 3: Sanctum token based authentication कैसे implement करते हैं?

उत्तर: User model में `HasApiTokens` trait लगाते हैं, login पर `$user->createToken('name')->plainTextToken` से token generate करते हैं जो client store करता है और हर request में `Authorization: Bearer {token}` header में भेजता है। Routes middleware('auth:sanctum') से protect होते हैं।

प्रश्न 4: API Resource classes क्यों use करते हैं?

उत्तर: `php artisan make:resource PostResource` से response की shape control होती है — जरूरी fields ही expose होते हैं, relationships conditionally load होते हैं (`whenLoaded`), और response format पूरे application में consistent रहता है, model directly return करने के बजाय।

प्रश्न 5: API rate limiting/throttling कैसे लगाते हैं?

उत्तर: `throttle:60,1` middleware (60 requests per minute) route पर लगा सकते हैं, या custom `RateLimiter::for()` rule बनाकर user-specific/plan-based limits define कर सकते हैं (`bootstrap/app.php` या `RouteServiceProvider` में)।

8. Validation & Form Requests

प्रश्न 1: Form Request class क्या फायदा देती है inline validate() के मुकाबले?

उत्तर: php artisan make:request StorePostRequest से बनी class validation rules, custom messages और authorization logic को Controller से अलग रखती है — code साफ रहता है, rules reusable होते हैं, और authorize() method से यह भी check हो जाता है कि user को यह action करने की permission है या नहीं।

प्रश्न 2: Custom validation rule कैसे बनाते हैं?

उत्तर: php artisan make:rule UniqueSlug से Rule class बनती है जिसमें passes() (या नए Laravel में validate() with closure) method में logic लिखते हैं, फिर rules array में new UniqueSlug() जोड़ते हैं। Simple cases के लिए Closure-based rule भी directly लिख सकते हैं।

प्रश्न 3: Conditional validation (sometimes, required_if) कैसे handle करते हैं?

उत्तर: 'field' => 'sometimes|required' से field तभी validate होती है जब request में present हो। required_if:type,premium जैसे rules से किसी दूसरे field की value पर depend करती हुई validation लिख सकते हैं।

प्रश्न 4: Nested array validation कैसे करते हैं (जैसे items का array)?

उत्तर: 'items' => 'required|array', 'items.*.name' => 'required|string', 'items.*.price' => 'required|numeric' जैसे dot notation से हर array element के अंदर के fields validate होते हैं।

9. Events & Listeners

प्रश्न 1: Events और Listeners क्यों use करते हैं?

उत्तर: यह code को decouple करने का तरीका है — जैसे OrderShipped event fire होने पर कई अलग-अलग Listeners (email भेजना, SMS भेजना, inventory update करना) independently react कर सकते हैं, बिना Controller में सब कुछ hardcode किए।

प्रश्न 2: Event listener को queue पर कैसे चलाएँ?

उत्तर: Listener class में ShouldQueue interface implement करने से Laravel automatically उसे queue पर dispatch करता है, बिना extra code के। इससे heavy काम (email/notification भेजना) request को block नहीं करते।

प्रश्न 3: Model Events और custom Events में फर्क बताइए।

उत्तर: Model Events (creating, saved आदि) Eloquent lifecycle से automatically जुड़े होते हैं। Custom Events business logic के specific points पर आप खुद define और fire (event(new OrderPlaced(\$order))) करते हैं — जैसे 'order placed हुआ', 'user upgraded to premium'।

प्रश्न 4: Real scenario: User registration के बाद welcome email, admin notification और CRM sync — कैसे design करेंगे?

उत्तर: UserRegistered event fire करेंगे registration के बाद, और तीन अलग Listeners बनाएँगे: SendWelcomeEmail, NotifyAdmin, SyncToCrm — सभी ShouldQueue implement करेंगे ताकि registration response तुरंत मिले और बाकी काम background में हो, और एक feature fail होने पर बाकी पर असर न पड़े।

10. Jobs & Queues

प्रश्न 1: Job और Queue का basic concept समझाइए।

उत्तर: Job एक class होती है (php artisan make:job) जिसमें कोई time-consuming task का logic handle() method में लिखा होता है। Queue उस Job को background में process करने के लिए एक ordered list/pipeline है, जिसे Worker process उठाकर execute करता है — जिससे user को response तुरंत मिल जाता है।

प्रश्न 2: Queue Driver options (sync, database, redis) में फर्क बताइए।

उत्तर: sync driver जॉब को तुरंत ही, उसी request में चला देता है (testing के लिए)। database driver jobs table में store करता है। redis driver production के लिए सबसे common है — fast, reliable और Horizon जैसे tools के साथ अच्छा काम करता है।

प्रश्न 3: Failed jobs कैसे handle करते हैं?

उत्तर: failed_jobs table (php artisan queue:table के बाद migrate) में failed jobs store होते हैं। Job class में failed() method लिखकर custom failure logic (जैसे admin को notify करना) डाल सकते हैं। php artisan queue:retry से दोबारा try कर सकते हैं। \$tries और backoff() property से retry attempts और delay control होता है।

प्रश्न 4: Laravel Horizon क्या है और क्यों use करते हैं?

उत्तर: Horizon Redis queues के लिए एक dashboard और configuration system है — real-time में जॉब्स का status, throughput, failed jobs देख सकते हैं, और supervisors/worker processes को config file से manage कर सकते हैं, बिना manually कई worker commands चलाए।

प्रश्न 5: Real scenario: bulk email भेजना 10,000 users को — कैसे design करेंगे?

उत्तर: एक बड़ा single Job बनाने के बजाय Job Chaining/Batching (Bus::batch()) use करेंगे — users को chunks में तोड़कर हर chunk का अलग Job बनाएँगे, ताकि parallel processing हो, कोई एक failure पूरे batch को block न करे, और progress track किया जा सके।

11. Redis & Cache

प्रश्न 1: Redis को Laravel में किन-किन कार्यों के लिए use करते हैं?

उत्तर: Cache driver (query results, computed data store करना), Session driver, Queue driver, और real-time features (pub/sub, rate limiting counters) के लिए — यानी fast in-memory key-value store की जरूरत जहाँ भी हो।

प्रश्न 2: `Cache::remember()` कैसे काम करता है?

उत्तर: `Cache::remember('key', $seconds, function(){ return expensive_query(); })` पहली बार closure चलाकर result cache करता है, अगली calls में cache से directly value return होती है जब तक expire न हो — इससे बार-बार heavy query/computation नहीं चलानी पड़ती।

प्रश्न 3: Cache invalidation कैसे handle करेंगे जब underlying data update हो?

उत्तर: Model के saved/deleted events पर या Observer में related cache keys को `Cache::forget()` से clear करते हैं, या cache tags (`Cache::tags(['posts'])->flush()`) use करते हैं ताकि related सारी cache entries एक साथ invalidate हो जाएँ — stale data दिखने से बचने के लिए यह जरूरी है।

प्रश्न 4: Session और Cache दोनों के लिए Redis use करने पर क्या ध्यान रखेंगे?

उत्तर: अलग-अलग Redis databases/connections (`config/database.php` में) या key prefixes use करते हैं ताकि session data और cache data आपस में conflict न करें, और production में persistence (RDB/AOF) properly configure करते हैं ताकि restart पर sessions loss न हों।

12. Laravel Scheduler & Cron

प्रश्न 1: Laravel Scheduler कैसे काम करता है?

उत्तर: एक single server cron entry (`* * * * * php artisan schedule:run`) हर minute चलता है, और `routes/console.php` (या पुराने `Kernel.php` में) define किए गए schedules के हिसाब से check करता है कि किसी task का time हुआ है या नहीं — इससे server पर सिर्फ एक cron entry manage करनी पड़ती है, कई अलग cron jobs नहीं।

प्रश्न 2: `withoutOverlapping()` और `onOneServer()` का use case क्या है?

उत्तर: `withoutOverlapping()` तब use करते हैं जब एक task पूरा execute होने से पहले उसका अगला run शुरू न हो (जैसे long-running report)। `onOneServer()` multi-server/load-balanced setup में जरूरी है ताकि एक ही scheduled task कई servers पर एक साथ न चल जाए।

प्रश्न 3: Scheduled task fail हो जाए तो कैसे पता चलेगा (monitoring)?

उत्तर: `->emailOutputOnFailure()` या `->onFailure(callback)` attach कर सकते हैं, logs check कर सकते हैं, या third-party monitoring (जैसे `healthchecks.io` ping) integrate कर सकते हैं ताकि silent failures पकड़ में आएँ।

13. Service Container & Dependency Injection

प्रश्न 1: Service Container (IoC Container) क्या है?

उत्तर: यह एक powerful tool है जो class dependencies को manage और resolve करता है। जब भी किसी class को उसकी dependency चाहिए (constructor में type-hint करने पर), container automatically उसका instance बनाकर inject कर देता है — इससे classes loosely coupled और testable रहती हैं।

प्रश्न 2: bind() और singleton() में क्या फर्क है?

उत्तर: bind() हर बार resolve होने पर नया instance बनाता है। singleton() पहली बार resolve होने पर instance बनाकर cache कर लेता है, बाद की सभी calls पर वही instance return होता है — जैसे कोई connection object जो पूरे request cycle में एक ही रहना चाहिए।

प्रश्न 3: Interface को concrete implementation से कैसे bind करते हैं (real example)?

उत्तर: AppServiceProvider के register() में `$this->app->bind(PaymentGatewayInterface::class, StripeGateway::class)` लिखते हैं। इससे Controller में PaymentGatewayInterface type-hint करने पर Laravel automatically StripeGateway inject करेगा — कल Razorpay में switch करना हो तो सिर्फ binding बदलनी पड़ेगी, Controller code नहीं।

प्रश्न 4: Constructor injection vs Method injection कब use करेंगे?

उत्तर: Constructor injection उन dependencies के लिए जो class में हर जगह चाहिए (जैसे Repository)। Method injection (Controller methods में type-hint) उन dependencies के लिए जो सिर्फ एक specific action में चाहिए, जैसे Request object।

14. Service Provider & Facades

प्रश्न 1: Service Provider क्या करता है और register() vs boot() में फर्क?

उत्तर: Service Provider application bootstrapping का central place है। register() में सिर्फ bindings register करते हैं (कोई दूसरा service resolve नहीं करना चाहिए यहाँ)। boot() तब चलता है जब सारे providers register हो चुके होते हैं, यहाँ routes, views, event listeners आदि register कर सकते हैं क्योंकि अब सभी dependencies available हैं।

प्रश्न 2: Custom Service Provider कैसे और कब बनाएँगे?

उत्तर: php artisan make:provider PaymentServiceProvider से बनाकर config/app.php (या bootstrap/providers.php Laravel 11+ में) में register करते हैं। जब कोई package/module का अपना setup logic हो (bindings, config publish, migrations load) तब अलग provider बनाना अच्छा practice है।

प्रश्न 3: Facade असल में कैसे काम करता है (internally)?

उत्तर: Facade एक static-looking interface है लेकिन असल में यह Service Container से underlying object resolve करके उस पर method call करता है। हर Facade class में getFacadeAccessor() method होता है जो बताता है कि container में कौन सा binding resolve करनी है।

प्रश्न 4: अपना खुद का Facade कैसे बनाएँगे?

उत्तर: पहले Service Provider में binding register करेंगे, फिर एक class बनाएँगे जो Illuminate\Support\Facades\Facade extend करेगी और getFacadeAccessor() में binding key return करेगी — फिर उसे config/app.php के aliases (या manual import) से use कर सकते हैं।

15. Transactions & Exception Handling

प्रश्न 1: Database Transactions कब जरूरी होते हैं, example दीजिए?

उत्तर: जब एक साथ कई related database operations हों और सब या तो पूरे succeed हों या सब rollback हों — जैसे order create करना और साथ ही उसकी inventory कम करना और payment record बनाना। `DB::transaction(function(){...})` या manual `DB::beginTransaction()/commit()/rollBack()` से implement करते हैं।

प्रश्न 2: Nested transactions और savepoints कैसे handle होते हैं?

उत्तर: Laravel MySQL में nested `DB::transaction()` calls को savepoints की मदद से handle करता है — अगर inner transaction fail होता है तो सिर्फ उस part तक rollback होता है, outer transaction को पूरी तरह affect नहीं करता, बशर्ते database savepoints support करे।

प्रश्न 3: Global exception handling कैसे customize करते हैं (Laravel 11 style)?

उत्तर: `bootstrap/app.php` में `withExceptions(function (Exceptions $exceptions) {...})` block में `$exceptions->render()` से specific exception types के लिए custom response define कर सकते हैं, जैसे API के लिए हमेशा JSON error response देना, HTML page नहीं।

प्रश्न 4: Custom Exception class कब बनाएँगे?

उत्तर: जब कोई specific business error हो जिसे meaningfully handle करना हो — जैसे `InsufficientBalanceException`। इससे catch blocks में specific handling हो सकती है, और Exception class के अंदर ही `report()` या `render()` method से logging/response logic रख सकते हैं।

16. N+ 1 Query & Eager Loading

प्रश्न 1: N+ 1 query problem क्या है, example से समझाइए?

उत्तर: मान लीजिए 100 Posts हैं और हर Post का Author दिखाना है — `Post::all()` से 1 query चलती है, फिर हर post के लिए `$post->author` access करने पर अलग-अलग query चलती है यानी 100 extra queries — कुल 101 queries। यही N+ 1 problem है, जो बड़े datasets में performance बुरी तरह गिरा देता है।

प्रश्न 2: Eager Loading से N+ 1 कैसे solve करते हैं?

उत्तर: `Post::with('author')->get()` इस्तेमाल करने से सिर्फ 2 queries चलती हैं — एक सारे posts के लिए, एक सारे related authors के लिए एक साथ (`WHERE id IN (...)`)। Nested relationships के लिए `with('author.company')` और multiple के लिए `with(['author', 'comments'])` use करते हैं।

प्रश्न 3: N+ 1 problem को कैसे detect करेंगे (tools)?

उत्तर: Laravel Debugbar या Telescope में queries count देखकर पता चलता है। `Model::preventLazyLoading()` (AppServiceProvider में, non-production में) लगाने से lazy loading होते ही exception throw होती है जिससे development में ही पकड़ में आ जाता है।

प्रश्न 4: Lazy Eager Loading (`load()`) कब use करेंगे `with()` की जगह?

उत्तर: जब आपके पास पहले से एक collection/model है और conditionally किसी relationship को load करना है — जैसे `if($showComments) { $posts->load('comments'); }` — यह query build होने के बाद relationship attach करता है।

17. Payment Gateway / Webhooks

प्रश्न 1: Payment gateway (Razorpay/Stripe) integration में मुख्य steps क्या हैं?

उत्तर: 1) SDK/package install करना, 2) API keys config में रखना (env से), 3) Order/payment intent create करना backend से, 4) frontend पर checkout flow दिखाना, 5) payment success के बाद server-side verification करना (signature verify), 6) webhook से final confirmation लेना — सिर्फ frontend callback पर भरोसा नहीं करना क्योंकि वो manipulate हो सकता है।

प्रश्न 2: Webhook signature verification क्यों जरूरी है?

उत्तर: यह ensure करता है कि request असल में payment provider से आई है, किसी fake/malicious request से नहीं। Provider हर webhook के साथ एक secret key से बना signature भेजता है, जिसे आप अपनी तरफ से secret के साथ recompute करके match करते हैं — mismatch होने पर request reject कर देते हैं।

प्रश्न 3: Webhook idempotency कैसे handle करेंगे (duplicate events से बचना)?

उत्तर: हर webhook event का एक unique event_id store करेंगे (एक processed_webhooks table में) और नया event process करने से पहले check करेंगे कि वह पहले process हो चुका है या नहीं — क्योंकि providers कभी-कभी same event दोबारा भेज सकते हैं (retry mechanism की वजह से)।

प्रश्न 4: Payment fail होने पर या webhook late आने पर order status कैसे manage करेंगे?

उत्तर: Order को शुरू में 'pending' status में रखेंगे, payment initiate होते ही नहीं बल्कि webhook से confirm होने पर ही 'paid' mark करेंगे। साथ ही एक scheduled job रखेंगे जो लंबे समय से pending orders को check/reconcile करे ताकि webhook miss होने पर भी system consistent रहे।

18. File Upload & Storage

प्रश्न 1: Laravel Storage facade और filesystem disks कैसे काम करते हैं?

उत्तर: config/filesystems.php में अलग-अलग disks (local, public, s3) define होती हैं। \$request->file('photo')->store('avatars', 's3') से specific disk पर file save होती है — इससे code same रहता है चाहे local storage use करें या S3, बस config बदलनी पड़ती है।

प्रश्न 2: File upload validation में क्या-क्या check करना चाहिए?

उत्तर: 'photo' => 'required|file|mimes:jpg,png,pdf|max:2048' जैसे rules से file type और size restrict करते हैं। साथ ही filename को खुद generate करना चाहिए (uniqid/hash से) ताकि malicious filenames से बचा जा सके, original filename पर भरोसा नहीं करना चाहिए।

प्रश्न 3: Large file upload (जैसे videos) को कैसे handle करेंगे?

उत्तर: Chunked upload (client-side में file को टुकड़ों में भेजना) use करते हैं ताकि timeout और memory issues न हों। Upload के बाद actual processing (जैसे video transcoding) को Queue Job में डालते हैं ताकि request block न हो।

प्रश्न 4: S3 पर सीधे browser से upload (direct upload) कैसे काम करता है?

उत्तर: Backend से एक pre-signed URL generate करते हैं (temporary, limited-time upload permission के साथ), frontend उस URL पर सीधे S3 को file भेजता है — इससे बड़ी files server से होकर नहीं गुजरती, server का bandwidth/memory बचता है।

19. Security

प्रश्न 1: Laravel में SQL Injection से कैसे बचाव होता है?

उत्तर: Eloquent और Query Builder parameter binding automatically करते हैं (prepared statements), इसलिए normal use में SQL injection का खतरा नहीं होता। सिर्फ raw queries (DB::raw, whereRaw) में खुद bindings ([...]) use करनी पड़ती हैं, कभी भी user input को directly string में concatenate नहीं करना चाहिए।

प्रश्न 2: CSRF Protection कैसे काम करता है?

उत्तर: हर session के लिए एक unique CSRF token generate होता है, जो forms में @csrf directive से hidden field में जाता है। हर state-changing request (POST/PUT/DELETE) पर VerifyCsrfToken middleware इस token को verify करता है ताकि किसी दूसरी malicious site से बिना जाने request न भेजी जा सके।

प्रश्न 3: XSS (Cross-Site Scripting) से बचाव कैसे करते हैं?

उत्तर: Blade का {{ \$variable }} syntax automatically output को escape करता है, इसलिए raw HTML/script tags execute नहीं होते। {!! !!} सिर्फ तभी use करें जब content पूरी तरह trusted हो, user input के साथ कभी नहीं।

प्रश्न 4: Mass Assignment vulnerability क्या है और कैसे रोकते हैं?

उत्तर: अगर Model में \$fillable define नहीं किया और create()/update() में सीधे \$request->all() pass कर दिया, तो user कोई भी field (जैसे is_admin) inject कर सकता है। \$fillable (whitelist) या \$guarded define करके सिर्फ allowed fields ही mass-assign होने देनी चाहिए।

प्रश्न 5: API security के लिए कौन-कौन सी practices follow करेंगे?

उत्तर: HTTPS हमेशा use करना, rate limiting लगाना, sensitive data (.env keys) कभी code में hardcode न करना, tokens की expiry रखना, input validation strict रखना, और CORS को सही तरीके से configure करना (सिर्फ trusted origins allow करना)।

प्रश्न 1: Git branching strategy (Git Flow) real project में कैसे follow करते हैं?

उत्तर: main/master हमेशा production-ready code रखता है, develop branch active development के लिए, हर feature के लिए feature/xyz branch बनाते हैं, release से पहले release/x.x branch, और urgent fixes के लिए hotfix/xyz — फिर PR/Merge Request से review के बाद merge करते हैं।

प्रश्न 2: Merge conflict कैसे resolve करते हैं?

उत्तर: git status से conflicted files देखते हैं, files में <<<<<< ===== >>>>>> markers के बीच manually सही code रखते हैं, फिर git add करके git commit (merge के लिए) या git rebase --continue (rebase के लिए) करते हैं।

प्रश्न 3: rebase और merge में क्या फर्क है, कब क्या use करेंगे?

उत्तर: merge दोनों branches के history को जोड़ता है और एक merge commit बनाता है — history में branching दिखती है। rebase commits को नए base पर 'replay' करता है जिससे history linear/clean रहती है। Shared/public branches पर rebase से बचना चाहिए क्योंकि यह history rewrite करता है।

प्रश्न 4: git cherry-pick और git stash किस scenario में use करते हैं?

उत्तर: cherry-pick तब use करते हैं जब एक specific commit को बिना पूरी branch merge किए दूसरी branch में लाना हो (जैसे urgent hotfix)। stash तब जब काम अधूरा हो और branch switch करनी पड़े — बिना commit किए changes temporarily save कर सकते हैं और बाद में git stash pop से वापस ला सकते हैं।

21. Laravel Performance Optimization

प्रश्न 1: Production deployment में कौन-कौन से cache commands जरूर चलाने चाहिए?

उत्तर: php artisan config:cache, route:cache, view:cache, और event:cache — ये सब application boot time को काफी कम कर देते हैं क्योंकि files हर बार parse नहीं होतीं, एक single compiled file से load होती हैं। ध्यान रहे config:cache के बाद env() सिर्फ config files में ही काम करता है, कोड में सीधे नहीं।

प्रश्न 2: Composer autoloader को कैसे optimize करते हैं?

उत्तर: composer install --optimize-autoloader --no-dev production deployment में चलाते हैं — यह class map को optimize करता है (classmap authoritative) जिससे class loading fast होती है और dev dependencies शामिल नहीं होतीं।

प्रश्न 3: Database query performance कैसे improve करेंगे बड़े dataset पर?

उत्तर: Eager loading से N+ 1 हटाना, जरूरी columns ही select करना, chunk()/cursor() से बड़ा data process करते वक्त memory बचाना, indexes लगाना, और read-heavy operations के लिए caching या read replicas इस्तेमाल करना।

प्रश्न 4: Queue और Cache के अलावा और कौन से performance techniques common हैं?

उत्तर: OPcache enable करना (PHP bytecode caching), CDN use करना static assets के लिए, image optimization, Laravel Octane use करना (long-running worker processes के साथ) high-traffic applications के लिए, और lazy loading images/assets frontend पर।

22. Real-world Scenario Questions

प्रश्न 1: अगर एक API endpoint production में अचानक slow हो जाए, आप कैसे debug करेंगे?

उत्तर: पहले Laravel Telescope/logs से check करेंगे कि कौन सी query/external call time ले रही है। Database slow query log, server resource usage (CPU/memory), और recent deployments/data growth check करेंगे। अगर external API call की वजह है तो timeout/circuit breaker pattern लगाने पर विचार करेंगे।

प्रश्न 2: एक large CSV file (लाखों rows) import करनी है, कैसे approach करेंगे?

उत्तर: पूरी file memory में load नहीं करेंगे — LazyCollection या chunk-by-chunk reading use करेंगे। Import को Queue Job में डालेंगे ताकि request timeout न हो, database inserts को chunk insert (insert() with array) से batch में करेंगे न कि row-by-row save(), और progress track करने के लिए एक status/progress column रखेंगे।

प्रश्न 3: दो microservices/systems के बीच data sync करना है (जैसे order system और inventory system), कैसे design करेंगे?

उत्तर: Events और Queue based approach अच्छा रहेगा — order create होने पर event fire होगा, एक Listener उसे queue job के through दूसरे system को API call या message queue (जैसे RabbitMQ/SQS) से भेजेगा। Failure की स्थिति में retry mechanism और idempotency keys रखेंगे ताकि duplicate processing न हो।

प्रश्न 4: अगर आपको legacy Laravel 6 application को Laravel 11 पर upgrade करना पड़े, approach क्या होगी?

उत्तर: पहले composer.json में एक-एक major version step-by-step upgrade करेंगे (6→7→8...→11), हर step पर Laravel के official upgrade guide follow करेंगे, deprecated features और breaking changes नोट करेंगे, अच्छी test coverage सुनिश्चित करेंगे ताकि हर step के बाद regression test हो सके, और directly production पर नहीं बल्कि staging environment में पहले पूरा process करेंगे।

23. 5–Year Experience Tricky Questions

प्रश्न 1: Laravel में Repository Pattern use करना चाहिए या नहीं — आपकी राय?

उत्तर: Eloquent खुद ही Active Record + Repository जैसे patterns को काफी हद तक cover करता है, तो छोटे/medium projects में extra Repository layer सिर्फ boilerplate बढ़ा सकती है। लेकिन अगर database swap करने की संभावना हो, testing में heavy mocking चाहिए हो, या business logic बहुत complex हो तो Repository/Service layer valuable होती है — यह team और project size पर depend करता है, कोई एक सही जवाब नहीं है।

प्रश्न 2: Queue worker crash हो जाए तो in–progress jobs का क्या होगा?

उत्तर: अगर job अभी तक commit/complete नहीं हुआ और worker crash हो गया, तो Redis/database driver में job फिर से visible हो जाता है (visibility timeout के बाद) और दोबारा process होता है — इसलिए jobs को idempotent design करना जरूरी है ताकि दोबारा चलने पर गलत result (जैसे double charge) न आए।

प्रश्न 3: Eager Loading हमेशा बेहतर होती है क्या, या इसकी भी limitation है?

उत्तर: नहीं, हमेशा नहीं — अगर relationship actually use ही नहीं होने वाली या सिर्फ कभी–कभार चाहिए, तो unnecessary eager loading extra memory/data load कर देगी। बहुत बड़े related datasets के लिए eager loading से भी memory issue हो सकता है, वहाँ chunk() या lazy loading with select specific columns better रहता है।

प्रश्न 4: Transaction के अंदर queue job dispatch करना क्यों risky है?

उत्तर: अगर transaction commit होने से पहले ही job queue पर चला जाए (खासकर sync/fast queue driver के साथ) और job अंदर वो data पढ़ने की कोशिश करे जो अभी तक commit नहीं हुआ, तो data not found जैसी errors आ सकती हैं। इसलिए afterCommit() (job में) या DB::afterCommit() callback use करना सही practice है।

प्रश्न 5: Cache का इस्तेमाल हर जगह करना चाहिए क्या performance के लिए?

उत्तर: नहीं — cache stale/inconsistent data का risk लाता है, invalidation logic जटिल हो सकती है (cache invalidation कंप्यूटर साइंस की मशहूर 'hard problems' में से एक है), और frequently–changing data के लिए cache का फायदा कम, complexity ज्यादा हो सकती है। सिर्फ उन जगहों पर cache करें जहाँ read बहुत ज्यादा हो और data अपेक्षाकृत stable हो।

24. Project–Based Questions

प्रश्न 1: अपने सबसे challenging project के बारे में बताइए — किस तरह के सवाल पूछे जा सकते हैं।

उत्तर: Interviewer यह जानना चाहता है: आपने किस problem को solve किया, क्यों वो approach चुनी (alternatives क्या थे), scale/performance की क्या challenges आई, team में आपका role क्या था, और आपने कैसे decisions लिए — technical depth के साथ-साथ communication भी test होती है।

प्रश्न 2: आपने अपने project में performance कैसे improve की — concrete example दीजिए (तैयार रखें)।

उत्तर: एक अच्छा जवाब structure यूँ होगा: 'Problem क्या था (जैसे dashboard load होने में 8 सेकंड लगते थे) → Root cause क्या मिला (N+ 1 queries / missing index) → क्या किया (eager loading + composite index + caching) → Result क्या मिला (2 सेकंड में load, measurable improvement)'। हमेशा numbers/metrics के साथ बताना ज्यादा असरदार होता है।

प्रश्न 3: Code review में आप क्या-क्या देखते हैं (आपके project experience के हिसाब से)?

उत्तर: Business logic सही जगह है (Controller पतला, Model/Service में logic), N+ 1 queries न हों, proper validation हो, security concerns (mass assignment, SQL injection) cover हों, naming conventions और consistency, और क्या tests लिखे गए हैं या नहीं — यह बताना चाहिए कि आप सिर्फ 'काम करता है' नहीं बल्कि maintainability भी देखते हैं।

प्रश्न 4: Team में disagreement हो जाए technical approach पर (जैसे architecture decision) — आप कैसे handle करेंगे?

उत्तर: पहले अपना point data/reasoning के साथ रखेंगे (pros/cons, trade-offs), दूसरे के perspective को सुनेंगे, अगर संभव हो तो एक small POC (proof of concept) बनाकर actual comparison करेंगे, और अंततः team/lead के साथ मिलकर एक decision पर पहुँचेंगे — ego की बजाय project के फायदे को प्राथमिकता देंगे।